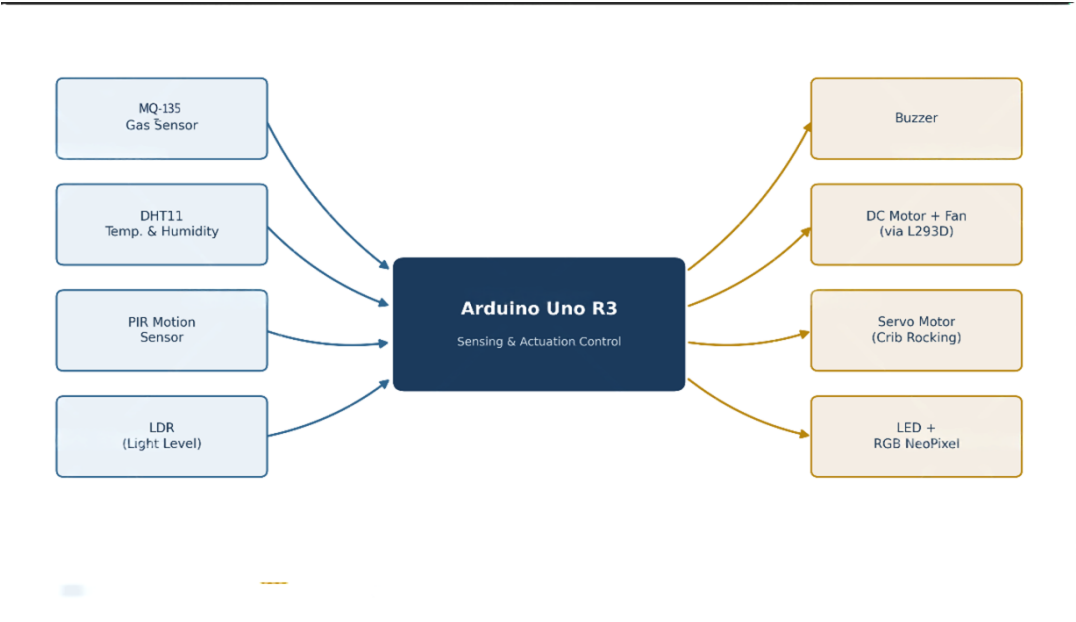


SMART NURSERY GUARDIAN

An AI-Assisted Embedded Monitoring System for Infant Care

HARDWARE SECTION — TECHNICAL REPORT

Software section included in a following section of this report



Project	Smart Nursery Guardian
Team	MindCloud
Scope of this document	Hardware design, simulation, and implementation
Date	September 12, 2026

1. General Project Overview

At three in the morning, a baby cries and a tired parent is left guessing why. A cry is an infant's only available language, and most caregivers cannot read it reliably under sleep deprivation and stress. **Smart Nursery Guardian** is our team's response to this problem: an embedded monitoring system that listens to a baby's cry, uses machine learning to infer its probable cause, and reacts within seconds — rocking the crib, displaying guidance to the caregiver, or raising a safety alert.

Beyond audio, the system continuously watches the nursery environment: motion near the crib, ambient light, temperature and humidity, and the presence of gas or smoke. Depending on the readings, it autonomously controls a room lamp, a cooling fan, and a status indicator, and it escalates to the caregiver's phone when a genuine safety condition is detected. The intent throughout is a system that behaves like an attentive caregiver who never sleeps and never misses a signal.

To keep the engineering work manageable and to match the skill sets on the team, the project is split into two cooperating parts:

- **Software section** — the cry-listening pipeline, the machine-learning classifier, the caregiver-facing interface, and the Telegram alerting logic, all running on a laptop.
- **Hardware section** — the sensing and actuation platform: the circuit design, the printed circuit board, the physical build, and the embedded firmware that reads sensors and drives actuators in real time.

The two sections communicate over a serial link: the hardware platform reports sensor events upward, and the software layer sends back classification results and control commands. This document reports on the **hardware section only** for now. The software section is being finalized separately and will be presented in a following section of this report once complete.

2. Hardware Section

The hardware section is responsible for turning the sensing and actuation requirements of the project into a working, well-documented physical platform: a schematic and PCB design produced in Altium Designer, two independent simulation passes for cross-verification, and a real, working prototype built from the components listed in the project requirements. The remainder of this document presents, in order, the development flow the team agreed on, the plan used to divide and execute the work, and the results obtained.

2.1 Development Flow

Rather than moving directly from schematic to physical board, the team adopted a five-stage flow that front-loads verification before any soldering happens, and then closes the loop with a second, more rigorous simulation pass after the physical build. Each stage's output is the direct input to the next.



Figure 1. The five-stage hardware development flow adopted for this project.

Stage	Purpose	Why it comes at this point
1. Altium Schematic	Capture every sensor and actuator connection to the Arduino Uno R3, assign pins, and define power distribution.	A precise schematic is the single source of truth the rest of the flow is checked against.
2. Tinkercad Simulation	Rebuild the circuit on a virtual breadboard using the schematic as the reference and confirm that the logic behaves as intended.	Functional mistakes are far cheaper to catch here than after a board is fabricated.
3. Altium PCB Layout	Convert the schematic — now functionally validated — into a routed, manufacturable 4-layer board with proper planes and JLCPCB-compatible design rules.	Layout is only worth doing once the underlying circuit is known to work.
4. Real Hardware Implementation	Assemble and power the physical prototype from the components listed in Section 2.3, wiring it exactly as validated in simulation.	This is the ultimate test: does the design survive contact with real components and real noise?
5. Proteus Simulation	Model the same circuit a second time in Proteus, a more rigorous, professional simulation environment, and compare its predictions against the real hardware's behavior.	Running this after the real build lets it double as an independent check on the physical results, not just a pre-build sanity test.

WHY TWO SIMULATORS

Tinkercad and Proteus serve different purposes in this flow rather than duplicating each other. Tinkercad is quick to set up and is used early to catch gross logic errors before committing to a PCB layout. Proteus offers more accurate, professional-grade electrical modeling and is used after the real implementation to rigorously cross-check the physical results — giving the team two independent points of comparison against the real board.

2.2 Work Plan

The hardware section was carried out by two team members working in a deliberate design-then-review pattern at every stage, so that no single piece of work reached the next stage unchecked.

Stage	Member A	Member B
Design capture	Creates the Altium schematic and the Altium PCB layout.	Reviews both against the requirements, runs ERC/DRC, and corrects any errors found.
Pre-build simulation	—	Builds the Tinkercad simulation directly from the validated Altium schematic to confirm functional behavior before fabrication.
Physical build	Carries out the real-world hardware implementation: assembly, wiring, and bring-up of the prototype.	—
Post-build verification	—	Reviews the real implementation and builds the Proteus simulation for a more professional, independent cross-check of the results.

This structure keeps design and verification in separate hands at every stage: the person who draws a schematic is not the one who signs off on it, and the person who builds the physical board is not the one who produces its independent simulation check. In practice this caught wiring and footprint mistakes early, before they became costly to fix in a fabricated board.

2.3 Components and System Architecture

The prototype is built around an Arduino Uno R3, which reads every sensor and drives every actuator in the hardware platform. Table 2 lists the components used and the role each plays in the system described in the project requirements.

Component	Role in the System
Arduino Uno R3	Central microcontroller: reads all sensors, runs the local control logic, drives all actuators, and relays sensor events to the software layer over serial.
MQ-135 Gas Sensor	Detects gas or smoke in the nursery; on threshold breach, triggers the system's highest-priority safety alert (buzzer and downstream phone notification).
DHT11 (Temperature & Humidity)	Monitors ambient temperature to drive the cooling fan and the status-color indicator, with humidity available as an additional environmental reading.
PIR Motion Sensor	Counts motion events near the crib; a burst of 4 or more motions within an 8-second window is interpreted as the baby being awake. <i>Note: in the real hardware implementation, this threshold was lowered to 2 motions within 8 seconds, because the physical PIR sensor takes approximately 3 seconds to fall from high back to low, limiting how many discrete motions it can register within that window.</i>
LDR (Photoresistor)	Measures ambient light level, used to decide whether the room lamp should be switched on.
LED (Room Lamp)	Provides room lighting, switched on automatically when the room is dark and the baby is awake or a cry is in progress.

Component	Role in the System
RGB NeoPixel (WS2812)	Gives an at-a-glance color status of the ambient temperature band (cool, moderate, warm).
DC Motor + Fan Blades	Provides active cooling, run at a speed proportional to the measured temperature. <i>Note: the maximum fan speed was capped at 230 (out of a maximum of 255) in the real hardware implementation, to limit the current the motor draws at its highest speed.</i>
L293D Motor Driver	Drives the DC fan motor at the current and voltage levels the Arduino's GPIO pins cannot supply directly.
Servo Motor	Physically rocks the crib the moment a cry is detected, as an immediate soothing response.
Buzzer	Provides audible alerts for urgent cries and for gas/smoke safety events.
Resistors & Capacitors	Support components for signal conditioning, pull-ups/pull-downs, and power-line decoupling across the sensor and driver stages.



Figure 2. Hardware system architecture: sensor inputs and actuator outputs around the Arduino Uno R3.

2.4 Results

The hardware platform was validated at three independent points: the Tinkercad simulation, the Proteus simulation, and the real physical build. Across all three, the observed behavior matched both the theoretical design expectations and the functional requirements defined in the project specification.

Behavior Checked	Expected (per requirements)	Outcome
Temperature response	Below 25°C: blue, fan off. 25–30°C: green, fan at half speed. Above 30°C: red, fan at full speed.	Matched across simulation and real hardware; color and fan speed always changed together from the same reading.

Behavior Checked	Expected (per requirements)	Outcome
Motion / "baby awake" logic	4 or more PIR motions within an 8-second window register as the baby being awake.	Threshold behaved as designed, correctly ignoring brief, isolated motion.
Room lamp logic	Lamp turns on only when the room is dark AND (baby awake OR a cry is in progress).	Combined condition verified correctly in both simulators and on the physical board.
Gas / smoke safety alert	Buzzer sounds continuously and the event is reported as the highest-priority alert.	Triggered reliably at the expected gas concentration threshold in all three verification points.
Crib-rocking response	Servo rocks continuously the moment a cry is confirmed.	Responded immediately in simulation and hardware, consistent with the intended soothing-first behavior.

Overall, the results obtained from the Tinkercad and Proteus simulations were consistent with each other and with the behavior observed on the real hardware, giving the team confidence that the design is sound both in theory and in practice. No discrepancy required a redesign of the schematic or the PCB layout; the changes made during the process were limited to the minor corrections identified during the schematic and layout review step described in Section 2.2.

STATUS

The hardware platform is complete, verified, and ready for integration. The next step is connecting this hardware to the software section over serial, so that cry classification results and sensor events can flow between the two halves of the system as one working whole.

This was the hardware section of Smart Nursery Guardian only. The software section — audio processing, the machine-learning classifier, the caregiver interface, and Telegram alerting — is being finalized separately and will be shown in the following section.

3. Software Section

The software section is the decision layer that turns raw audio and sensor data into concrete actions for the caregiver. It continuously listens to the nursery through a microphone, decides whether the captured sound is actually a baby crying, and if so classifies the cry as hungry, tired, or discomfort. It also communicates with the Arduino over serial to receive hardware alerts (gas/smoke, sleep/awake state, room temperature) and to drive the buzzer, shows all information on a desktop GUI, and additionally notifies the caregiver through a Telegram bot regardless of whether they are in front of the screen.

3.1 Architecture

The software is organized around one central Tkinter application (**GUI2.py**, class **SmartNurseryFullApp**) that starts three background threads when it launches: an **Arduino thread** that opens the serial port and continuously reads status lines while writing commands back; an **audio/ML thread** that captures microphone audio, runs the cry detector, extracts features, and calls the trained model; and a **Telegram thread** that runs the bot's polling loop independently of the GUI. None of these threads touch Tkinter widgets directly — every

UI update is scheduled with **root.after(0, ...)** so it always runs on the main thread, and the audio thread hands frames to the GUI through a `queue.Queue`. The ML model is trained separately in a notebook on the Donate-a-Cry corpus and exported with joblib as **cry_classifier_model.pkl**, which `GUI2.py` loads once at startup. Alerting is decoupled from detection: whenever a hungry, tired, or gas event happens, the GUI simply calls the matching function in `tele.py`, which broadcasts to every registered chat ID on its own thread.

Simplified data flow: Microphone → audio thread → amplitude/VAD gate → feature extraction → ML model → GUI update + Telegram alert, running alongside Arduino ↔ serial thread ↔ GUI (buzzer/video commands out, sensor readings in).

3.2 Core Components

Table 3 summarizes the main functional building blocks of the software and what each one does.

Component	What It Does
Audio Acquisition	Opens the microphone with <code>sounddevice</code> at its native sample rate; frames are queued by the stream callback, pulled into 2-second windows, resampled to 16 kHz, and denoised with <code>noisereduce</code> . A file-based source can substitute a fixed .wav for testing without a live microphone.
Cry / VAD Detection	A two-stage gate: a quiet chunk (peak amplitude below 0.06) is immediately treated as not-a-cry; otherwise the chunk is split into 20 ms frames and passed to <code>webrtcvad</code> , and the chunk is labelled a cry once the speech-frame ratio reaches 0.35. Every state change is mirrored to the Arduino as <code>CRY_START</code> / <code>CRY_STOP</code> .
Feature Extraction	Builds a 48-value feature vector per cry: 20 MFCC coefficients (mean/std, 300–600 Hz band), RMS energy, zero-crossing rate, and F0 statistics via <code>librosa.yin</code> — computed identically offline (training) and at runtime, so the model always sees features the same way.
ML Classification	Deployed model is a scikit-learn Pipeline (StandardScaler + SVC, RBF kernel, C=10, <code>class_weight="balanced"</code>), chosen by comparing a dummy baseline, SVM, Random Forest and Extra Trees under 10-fold GridSearchCV. Minority classes (tired, discomfort) were augmented on the training split only, via pitch shift, time stretch, noise, and high-pass filtering.
GUI	Tkinter window with a status bar (Arduino connection, baby state, temperature, audio status, model status), a debug line with live amplitude/cry flag, a central panel that switches to an embedded video player for calming clips, and a full-screen red overlay with a "Dismiss Alert" button for gas/smoke emergencies.
Serial Communication	<code>pyserial</code> at 115200 baud; <code>auto_detect_arduino_port()</code> scans COM ports for known USB-serial chip signatures. The thread reacts to <code>AWAKE</code> , <code>SLEEPING</code> , <code>GAS_ALERT</code> and <code>TEMP:<value></code> and writes <code>CRY_START</code> , <code>CRY_STOP</code> and <code>BUZZER_ON</code> back to the board.
Alerts / Notifications	<code>tele.py</code> runs a Telegram bot (<code>telebot</code>); <code>/start</code> registers a caregiver's chat ID in memory. <code>send_hungry_alert</code> , <code>send_tired_alert</code> and <code>send_gas_alert</code> broadcast to every registered chat, each wrapped in its own <code>try/except</code> so one failed delivery never blocks the rest.

3.3 Workflow

On launch, the GUI queries the audio device, loads the trained model, builds the interface, and starts the three background threads. The Telegram thread begins polling immediately; the Arduino thread keeps retrying

auto-detection until connected, then continuously updates the baby-state and temperature labels and reacts instantly to a GAS_ALERT line (red overlay, buzzer, Telegram alert). In parallel, the audio thread keeps grabbing 2-second chunks, denoising them, and running the amplitude + VAD check. Once a cry is detected, its features are classified and the result drives one of three actions: **hungry** plays a calming video and sends a Telegram alert; **tired** shows an urgent message and turns the buzzer on; **discomfort** shows a message asking the caregiver to check the diaper, clothing, or position (no Telegram alert for this case). Once the chunk is no longer classified as a cry, the pipeline returns to "Listening...". Closing the window stops all loops, releases the video capture, and closes the serial port cleanly.

3.4 Configuration and Dependencies

Table 4 lists the key runtime parameters in GUI2.py; Table 5 lists the main third-party libraries the software depends on.

Parameter	Value	Purpose
BAUD_RATE	115200	Serial link speed to the Arduino
SAMPLE_RATE	16000 Hz	Audio processing rate (chunks resampled to this)
CHUNK_DURATION	2 s	Length of each audio window analysed
AMP_THRESHOLD	0.06	Minimum peak amplitude before VAD is even run
CRY_FRAME_RATIO	0.35	Minimum fraction of speech frames to call a chunk a cry
VAD_AGGRESSIVENESS	3 (GUI) / 2 (train)	webrtcvad sensitivity level
AUDIO_SOURCE	"mic" or "file"	Selects live microphone or a fixed test .wav file

Library	Used For
tkinter	Desktop GUI
sounddevice	Microphone capture
librosa	Resampling; MFCC / RMS / ZCR / F0 feature extraction
webrtcvad	Voice activity detection
noisereduce	Background noise reduction
joblib / scikit-learn	Loading/saving and running the trained classification pipeline
numpy / pandas	Numerical processing and dataset handling
audiomentations	Audio augmentation for minority classes (training only)
opencv-python / Pillow	Reading and rendering the calming video
pyserial	Serial communication with the Arduino
pyTelegramBotAPI	Telegram bot and alert delivery

3.5 Model Deployment and Error Handling

At startup, the GUI tries `MODEL_PATH` and falls back to a lowercase filename; if neither exists, it still starts but shows "Model: File Missing" and skips prediction. When found, joblib restores the full pipeline and its class list and feature count are printed for diagnostics. At prediction time, the GUI compares its 48-value feature vector's length against the model's expected input and raises a clear error rather than silently mispredicting; the predicted label is matched against keyword lists (with numeric labels as fallback) so the GUI keeps working even if the model is retrained with slightly different label text. Table 6 summarizes how the main failure modes are handled.

Failure Mode	Handling
Missing / unreadable model file	UI shows "Model: File Missing" or "Model: Load Error"; prediction is skipped instead of crashing.
Feature / prediction errors	Caught inside the audio pipeline and shown in the main panel instead of stopping the thread.
Missing calming video	Checked with <code>os.path.exists</code> before playback; shows "Video file not found" instead of raising.
Audio device errors	Falls back to a default 48000 Hz rate if the device cannot be queried.
Serial port issues	Retries every 3 seconds, updates the connection label, and safely closes the port before retrying.
Serial write failures	Wrapped in try/except so a broken connection cannot crash the caller.
Telegram send failures	Each send is wrapped individually, so one bad chat ID does not block alerts to the rest.
General pipeline errors	A catch-all in the main loop logs the exception and sleeps 1 second before continuing, keeping the app alive.